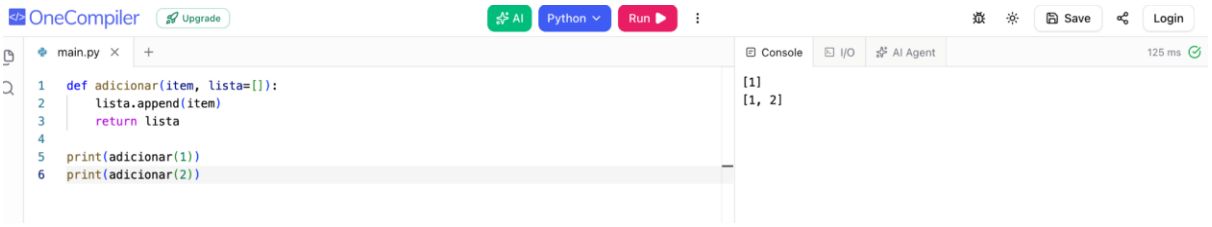
Paradigmas e Linguagens de Programação
Atividade aula-008 - Subprogramas

Sophia Machado Silva - RA: 24087451-2

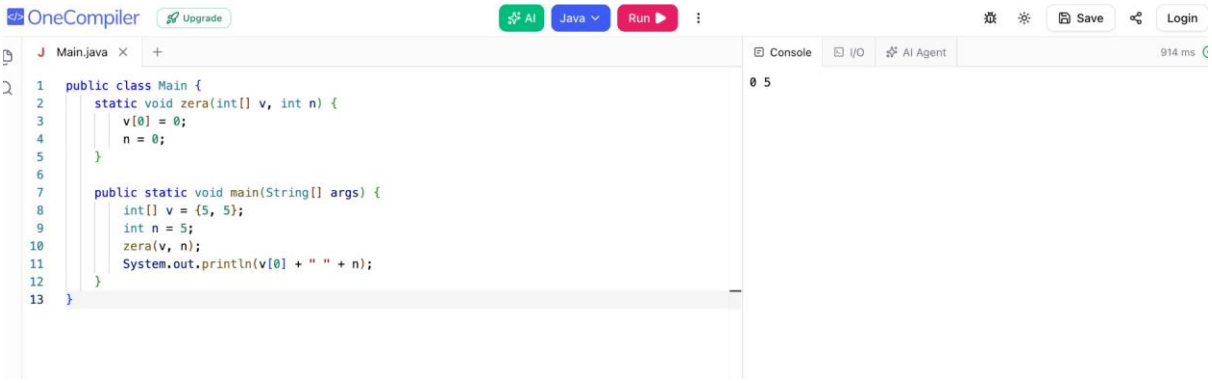
ex1: Python



a)

b) Parâmetro com valor padrão combinado com passagem por atribuição do Python. O valor padrão [ ] é criado uma única vez, na definição da função, e não a cada chamada. Como a lista é mutável, as duas chamadas compartilham o mesmo objeto (um apelido). Correção: lista=None e if lista is None: lista = [ ].

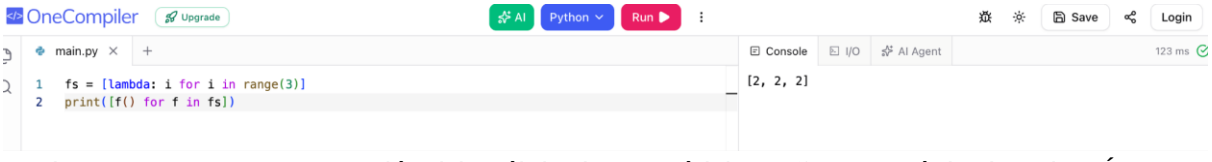
ex2: Java



a)

b) Em Java tudo é passado por valor. O n (primitivo) é uma cópia, então o chamador nunca vê a alteração. O v é uma cópia da referência, que aponta para o mesmo array, então v[0] = 0 altera o objeto compartilhado.

ex3: Python

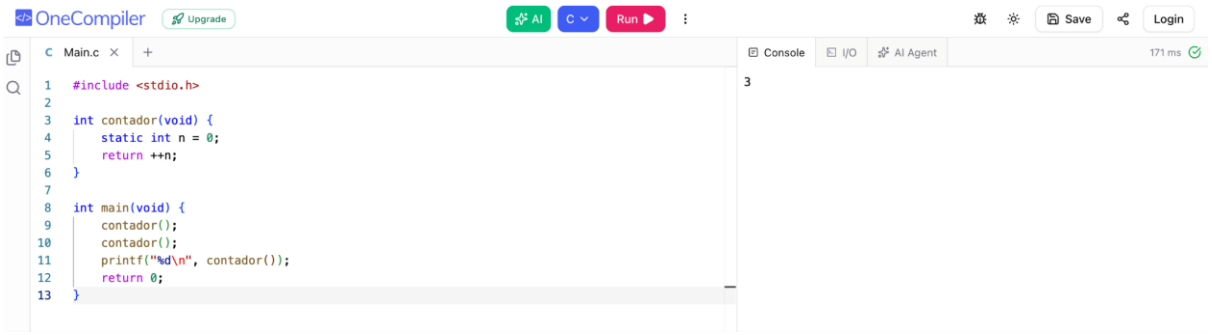


a)

b) Fechamentos capturam a variável (a célula de memória), e não uma cópia do valor. É o mesmo caso do var i em JavaScript, que dá [3, 3, 3]. Existe um único i, que vale 2 quando as lambdas são chamadas. Correção: lambda i=i: i, que dá [0, 1, 2].

ex4: C

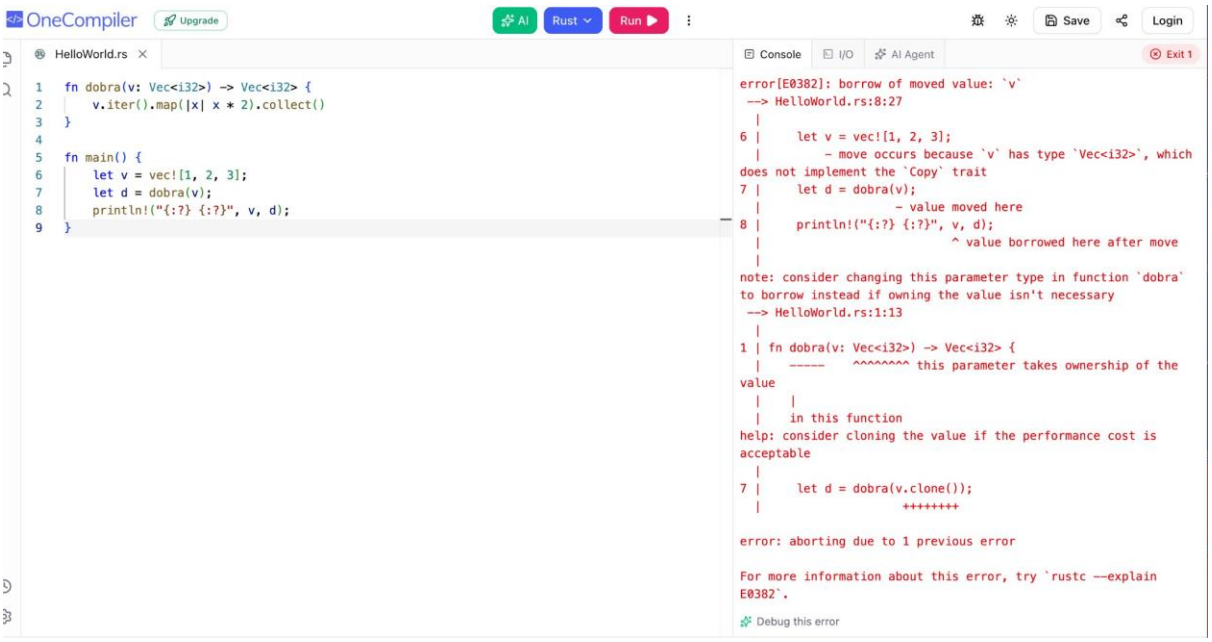
a)



b) variável local estática (static) nos ambientes de referenciamento local. Ela fica na área estática, e não na pilha, então lembra o valor entre as chamadas. Sem o static (dinâmica da pilha), a saída seria 1.

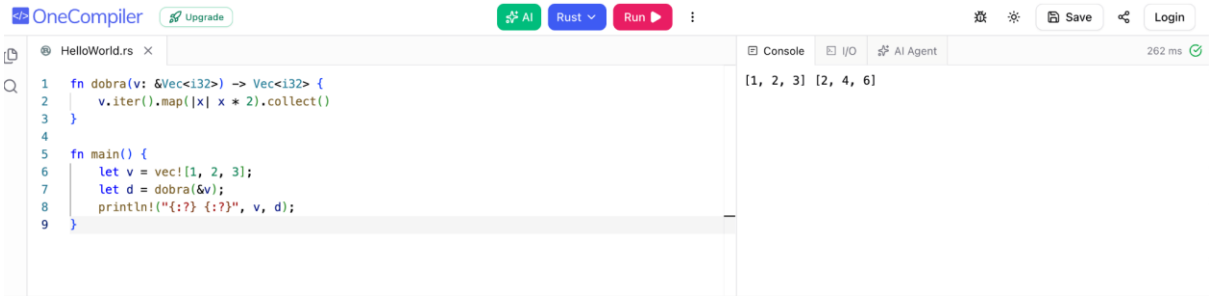
ex5: Rust

a)



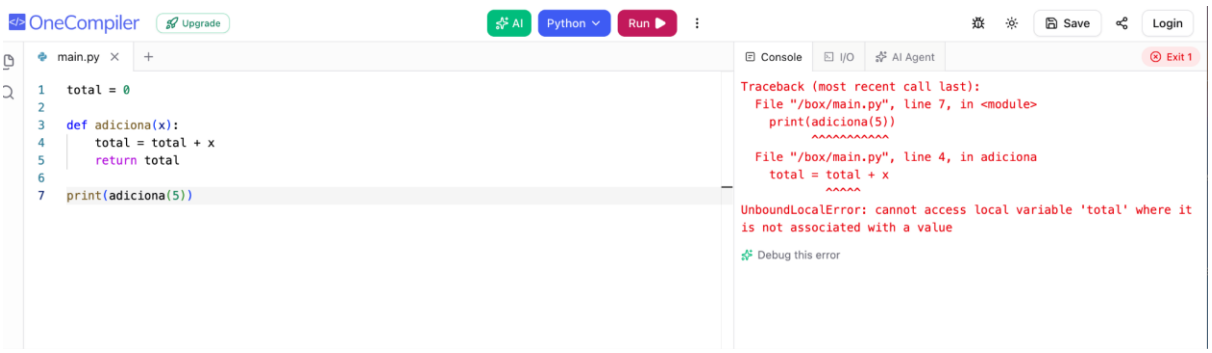
b) Em Rust a passagem padrão é por posse (move). Ao chamar dobra(v), a posse do vetor vai para a função e v deixa de ser válido no main. Usá-lo no println! é erro de compilação.

c) Versão corrigida:



ex6: Python

a)



b) Ambiente de referenciamento local. Como há uma atribuição a total dentro da função, o Python trata total como variável local em toda a função. Ao ler total + x, ela ainda não tem valor. Para alterar a global, declare global total no início da função. Em funções aninhadas, o equivalente é nonlocal (visto na síntese, na linha do Python). A alternativa sem efeito colateral é receber o total como parâmetro e devolver o novo valor.